

Detecting Silent Data Corruption From Hardware Counters

Minseop Choi

Dept. of Electrical & Computer Eng.

UMass Lowell

Lowell, USA

minseop_choi@student.uml.edu

Taha Azzaoui

Dept. of Electrical & Computer Eng.

UMass Lowell

Lowell, USA

taha_azzaoui@student.uml.edu

Kyle Chaisson

Dept. of Electrical & Computer Eng.

UMass Lowell

Lowell, USA

kyle_chaisson@student.uml.edu

Orlando Arias

Dept. of Electrical & Computer Eng.

UMass Lowell

Lowell, USA

orlando_arias@uml.edu

Seung Woo Son

Dept. of Electrical & Computer Eng.

UMass Lowell

Lowell, USA

seungwoo_son@uml.edu

Abstract—Silent Data Corruptions (SDCs), which can manifest at the application level despite extensive screening and testing, can disrupt meaningful scientific interpretation, thereby necessitating robust monitoring tools capable of detecting them. While prior approaches have demonstrated competitive detection performance, they often require nontrivial modifications to algorithms or prior knowledge, such as spatial or temporal data patterns, to make those approaches effective. Furthermore, the error model through standard random bit flips may not reflect realistic scenarios, potentially including relatively easy-to-detect cases with obvious deviations. In this work, we study SDCs and their effects on sparse matrix computations, prevalent kernels in many scientific applications, using hardware counters, which could serve as a holistic indicator of revealing program behavior changes due to SDCs. We experiment with a set of sparse matrix benchmarks using a method that simulates data corruption to varying degrees based on our extensive analysis of error propagation, creating realistic SDC occurrences at the application level. We detail the process of sampling hardware performance counters with minimal disturbance. Using the collected hardware counters, we train various classes of classifiers, including standard ML, neural-network-based, and unsupervised, to accurately detect SDCs. Our experimental evaluations through k-fold cross-validation indicate that hardware counters can effectively detect the presence of SDCs with a low false positive rate, incurring comparable training overheads and minor inference overhead compared to the state-of-the-art. Our approach achieves a competitive average recall (> 0.91) with a realistic error rate based on the observed error propagation and low runtime overhead ($< 2\%$) while avoiding program modifications.

Index Terms—Silent Data Corruption, Performance Monitoring Counters, Sparse Matrix-Vector Multiplication, Classification Algorithms, Error Injection

I. INTRODUCTION

Scientists and engineers conduct analyses that rely on extreme-scale simulations and AI, including machine learning (ML), to achieve breakthroughs in climate, energy, quantum physics, astrophysics, materials science, chemistry, biology, and more [35]. Therefore, the integrity of these large-scale systems and the trust scientists place in the data produced are

of utmost importance. As the complexity and density of large-scale systems increase, future large-scale systems and the data they generate, process, store, and transmit will be subject to increasingly higher data anomalies that are not well understood by the research community.

A soft error, namely silent data corruption (SDC), known to be caused by various factors, including hardware failures, electromagnetic interference, and power fluctuations, is of particular concern among various types of anomalies, which is much more prevalent than one may realize [9], [11], [20], [21], [31], [33], [36], [42], [43], [49], [54], [58]. For example, Jaguar, a petascale supercomputer at ORNL, suffers a double-bit memory error once every 24 hours [21]. A recent research study at Meta, Google, and Alibaba also confirmed that fleets of servers in data centers are experiencing SDC due to device characteristics inside hundreds of CPUs [16], [24], [52]. The situation only worsens: cosmic radiation can trigger a soft error, and simple down-to-earth factors such as temperature and power consumption can also be the culprit [36].

Further exacerbating the situation is the rapid emergence of ASIC accelerators for neural engines, which are prone to higher error rates than general-purpose computing hardware [58]. More importantly, this corrupted data, especially that which could alter results, may go undetected because current HPC software stacks largely lack mechanisms to inform scientists of data compromises. As the volume of data required for gathering and analysis increases, scientific discovery will be significantly hindered, as scientists will struggle to trust the data they generate and manipulate. This challenge could, in turn, negatively impact the integrity of their scientific interpretation [2], [8], [14], [41]–[43]. To ensure that scientists are aware of silent data compromises and to boost confidence in high-performance computing (HPC) systems, efficient and cost-effective data anomaly monitoring and detection mechanisms are essential.

Prior approaches to mitigate the impact of SDC within

large-scale systems are mostly architectural or at the system level [16], [24], algorithm level, or higher [10], [11], [26], [30], [32], [46]. Nonetheless, corrupted data would still be used by applications including the sparse matrix computations which underpin scientific applications, machine learning and AI workloads, and network analysis of significance [3], [7]. While the impact of SDC and its propagation can be monitored and tracked at the application layers effectively, adapting to each computational kernel or application is challenging due to the program modifications required by domain scientists. Such approaches also incur significant runtime overheads, as the monitoring (and detection) mechanism is part of the program execution. Traditional Algorithm-Based Fault Tolerance (ABFT) techniques [45], [47] rely on column checksums to detect and correct errors but introduce significant computational overhead. Online-ABFT [12] attempts to reduce this cost by monitoring mathematical properties during runtime, yet it remains algorithm-specific. As such, the need for a general-purpose low-overhead SDC detection framework is needed. Additionally, error models and the error injection mechanisms used in prior studies primarily focused on random bit flips, which is a standard and reasonable evaluation approach. However, it remains unclear whether the resulting error models will realistically translate into practice. For example, bit flips in the exponent can result in significant deviations from the original value, making detection relatively straightforward, while bit flips in the mantissa bits (e.g., any of the 32 least significant bits in double-precision numbers) introduce negligible errors that can be absorbed as part of the computations [40]. Therefore, the error injection module designed to evaluate SDC detection techniques requires a realistic error model.

Performance monitoring events (PMEs), which are typically supported by the performance monitoring unit (PMU) available in modern Intel or AMD CPUs [27], [28], [56] and even GPUs [37], have been primarily used to detect performance bottlenecks and potentially tune performance and energy/power consumption [56]. Recent years have witnessed an extended use of PMEs for other purposes, including SDC detection, as PMEs could be an instant signal for capturing the effect of an SDC [19], [23], [29]. While predicting SDC rates using existing PMEs is an interesting direction, prior studies have revealed a relatively weak relationship with only a few performance counters [23]. To our knowledge, no study has produced a model that learns from a holistic view of performance counters and validates such a model's detection performance on sparse matrix computations, which exhibit computational and data access patterns prevalent in many crucial scientific applications.

In this paper, we present an accurate and efficient SDC detection mechanism that utilizes PMEs to capture anomalous program behavior. To the best of our knowledge, this is the first systematic evaluation of the impact of SDC on critical HPC kernels using hardware performance monitoring counters. The main contributions of this paper are summarized as follows:

- We first provide an in-depth analysis of the compressed row storage (CRS) format designed to encode a sparse

matrix for efficient computation and examine how different corruption cases could affect the input of a sparse matrix operation, leading to corruptions in the row or column arrays or values. These realistic corruption cases ultimately result in changes to either memory access patterns or floating-point microarchitectural events, which we aim to exploit to detect SDCs.

- We present a novel approach for detecting SDC in sparse matrix computations using hardware performance counters. We develop a library for programming model-specific registers to efficiently collect hardware performance counters during execution. We comprehensively analyze the distribution of counters with and without SDCs rather than individually correlating with SDCs.
- We train seven classification algorithms (logistic regression, decision tree, random forest, k-nearest neighbor, multi-layer perception, gradient boosting, and restricted boltzmann machine) using the SuiteSparse Matrix Collection [13] and the HPCG benchmark suite [17], a widely used set of sparse matrix benchmarks.
- We evaluate our approach by injecting controlled errors into the program input (i.e., sparse matrices), small randomly selected data points with varying error rates to simulate realistic SDC occurrence (including rare, minor corruption). We demonstrate the effectiveness of our proposed approach in terms of detection accuracy and overheads incurred during training and inference.
- We conduct extensive experiments using a total of seven datasets from different sources with different classifiers, three from HPCG and four from the SuiteSparse Matrix Collection. Our framework achieves an accuracy of up to 94% and a recall of up to 96% when detecting data corruption while incurring extremely low runtime overhead, and crucially, without requiring modification to existing software.

II. BACKGROUND AND MOTIVATION

A. Sparse Matrix Computations

Sparse matrices are matrices where the majority of the elements are zero-valued. The sparsity coefficient s of a matrix $\mathbf{M} \in \mathbb{R}^{m \times n}$ with k non-zero values is given by

$$s = \frac{mn - k}{mn}, \quad (1)$$

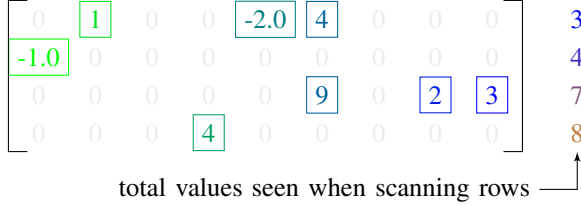
or the fraction of the total number of zeros over the total number of elements.

Sparse matrix-vector multiplication (SpMV) of the form $\mathbf{y} = \mathbf{A}\mathbf{x}$, where $\mathbf{A}$ is a sparse matrix (primarily zeros), is a fundamental computation in scientific computing, machine learning, and network analysis. The SpMV operation is memory bandwidth bound and, therefore, difficult to optimize. Optimizing the SpMV operation is an open research problem spanning (1) data structures for efficiently storing sparse matrices [22] (2) algorithmic improvements [57] (3) parallelization and matrix partitioning [6], [38], [53] (4) hardware acceleration [39], [59]. The following section describes the

compressed row storage format commonly used to reduce the memory footprint of a sparse matrix.

1) *Compressed Row Storage Format*: The Compressed Row Storage (CRS) format was designed to encode a sparse matrix and store it in memory while being able to efficiently perform computations with it [4]. The CRS format encodes a sparse matrix by storing all nonzero elements in a contiguous array. It consists of

- 1) a value vector $\mathbf{v} \in \mathbb{R}^k$ of nonzero values (row-wise),
- 2) a column index vector $\mathbf{c} \in \mathbb{Z}^k$, and
- 3) a row total vector $\mathbf{r} \in \mathbb{Z}^{m+1}$ where the r_i encodes the total number of nonzero values above row i .



(a) Sparse matrix with non-zero coefficients highlighted.

values $\mathbf{v}$: 1 -2.0 4 -1.0 9 2 3 4
column index $\mathbf{c}$: 1 4 5 0 5 7 8 3
row total $\mathbf{r}$: 0 3 4 7 8

(b) Resulting CRS representation of the matrix.

Fig. 1: CRS storage format. Three linear arrays encode the sparse matrix.

An example of the format is represented in Fig. 1. The values vector is read from the top left of the matrix going horizontally. The column number of the value is recorded as an entry on the column index vector. The number of elements encountered in the row is accumulated and appended to the row index vector. This results in three arrays, which can be contiguously stored in the memory of a computer.

This representation results in a smaller memory footprint while still providing efficient access to all elements in the matrix. One advantage of the CRS representation is that row slicing is efficient. In particular, the SpMV operation can be parallelized trivially. As matrix row operations are independent of each other, row-major computations and array lookups can be performed in parallel without the use of a synchronization primitive. This parallelism can be exploited in software implementations [5], [6], [44] as well as hardware implementations [1], [59], [60].

2) *SDC Propagation in Sparse Matrix Computation*: In this work, we study errors affecting the input data of a sparse matrix operation. Such errors may not necessarily affect the computation either because they are masked by overwriting or because they are detected by an error detection mechanism and subsequently recomputed. Others can go undetected, causing a runtime fault at best or an erroneous result at worst. We

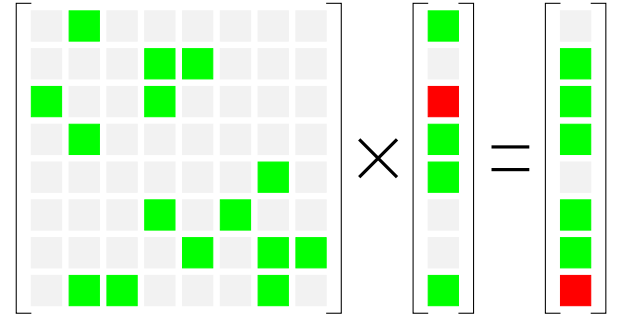


Fig. 2: Error Propagation in the SpMV Operation. Here, every nonzero value (in green color) in the third column will be affected by the corrupted value (in red color) in the third row of the dense operand. Further operations with the resulting vector will continue to propagate the error.

focus on the latter group of so-called silent errors. In the context of sparse matrix workloads, silent data corruption could potentially lead to inaccurate results if the values stored in either the sparse matrix or the dense operand are affected. Fig. 2 shows a specific data corruption pattern in sparse matrix-vector multiplication (SpMV), a primitive operation in sparse matrix computations, where a silent data corruption in the dense operand propagates to the result vector. A SpMV operation consists of consecutive dot products between the rows of the sparse matrix and the dense operand. In this example, every nonzero value in column i will be affected by the error in row i of the dense operand. In general, the impact of SDC on sparse matrix multiplication depends on (1) the specific data corruption patterns observed, (2) the size of the matrices being multiplied, and (3) the error detection mechanisms in place.

Detecting SDC in sparse matrix operations like SpMV is nontrivial due to the complexity at the instruction level. Prior works tend to compare individual loads and stores with known valid entries [7]. This has two main disadvantages. First, tracking individual instructions is prohibitively expensive as the input size grows. Second, the approach assumes that valid computational graphs are known beforehand. To improve on prior work, we instead look to the performance monitoring unit (PMU), which can be understood as a summary of the computation graph at runtime. We show that it is enough to study hardware performance counters to detect corruptions with significant accuracy. More importantly, performance counters do not depend on the input matrix size, a property which we exploit to achieve scalable SDC detection.

B. The Performance Monitoring Unit

The Performance Monitoring Unit (PMU) is a specialized hardware module in modern processors that enables detailed performance analysis for profiling. The PMU provides a set of hardware counters, which can be tied to hardware events such as cache misses, instruction cycles, page faults, and memory accesses. Microarchitectural events, such as the number of microoperations retired, can also be monitored. To use the

PMUs available in modern Intel CPUs, software must select which events to monitor by interfacing with a limited number of model-specific registers (MSRs). If more events need to be monitored than counter registers are available, software must schedule the utilization of the counters across the desired events¹. However, counter scheduling often leads to events being missed, as counters will not update the values of events while they are preempted. Intel CPUs allow for configuration of the PMU subsystem using the `rdmsr` and `wrmsr` instructions. Under normal operation, these instructions are privileged, however operating systems can provide mechanisms to access the PMU (using e.g., the control register `cr4`) at the cost of some performance overhead. This way, the operating system can set up the CPU to allow user processes to read from the PMU directly without trapping into the kernel using the `rdpmc` instruction.

III. TOWARDS A SDC DETECTION FRAMEWORK

The overall structure of our SDC detection framework is depicted in Fig. 3. Matrices are used as inputs to a sparse matrix operation that, when run, collects relevant hardware monitoring events. Once the computation terminates, the PMU values are read and used as input to a pre-trained model. The model then predicts whether SDC has occurred or not.

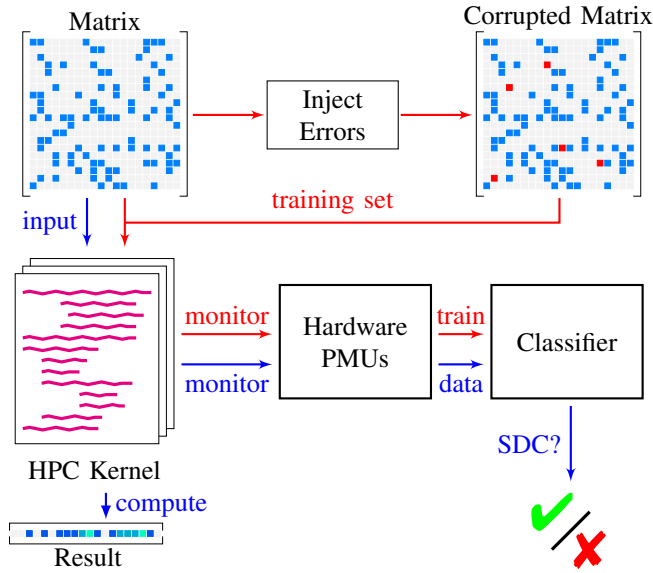


Fig. 3: Overview of the approach. We divide the approach in a training phase (→) and a deployment phase (→).

A. Foundations of the Methodology

We leverage microarchitectural advances in the development of high-performance CPUs currently being deployed in computing clusters to perform our analysis. By utilizing the performance monitoring subsystems of modern microarchitectures,

¹The number of available counters is dependent on the microarchitecture. For example, the Intel Skylake microarchitecture provides four general-purpose counters per logical core.

we are able to examine and infer characteristics about both the data flow within the CPU as well as the computations the CPU is performing to determine the likelihood of silent data corruption occurring in a given workload.

$$\begin{bmatrix} 0 & 1 & 0 & 0 & -2.0 & 4 & 0 & 0 & 0 \\ -1.0 & 0 & 0 & 0 & 0 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 2 & 3 & 0 \\ 0 & 0 & 0 & 4 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

v : 1 -2.0 4 -1.0 9 2 3 4
c : 1 4 5 0 5 7 8 3
r : 0 3 5 7 8

(a) Corruption on the rows array changes the row position of elements. Memory access patterns change as a result.

$$\begin{bmatrix} 0 & 1 & 0 & 0 & -2.0 & 4 & 0 & 0 & 0 \\ -1.0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 9 & 2 & 3 \\ 0 & 0 & 0 & 4 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

v : 1 -2.0 4 -1.0 9 2 3 4
c : 1 4 5 0 6 7 8 3
r : 0 3 4 7 8

(b) Corruption on the columns array changes the column position of elements. Memory access patterns change as a result.

$$\begin{bmatrix} 0 & 1 & 0 & 0 & -2.0 & 4 & 0 & 0 & 0 \\ -1.0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 9.1 & 2 & 3 \\ 0 & 0 & 0 & 4 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

v : 1 -2.0 4 -1.0 9.1 2 3 4
c : 1 4 5 0 5 7 8 3
r : 0 3 4 7 8

(c) Corruption on the values array changes the overall result of the computation. Floating-point microarchitectural events change as a result.

Fig. 4: Possible effects of SDC in CRS-stored matrices in (a) rows, (b) columns, and (c) values. The baseline matrix is shown in Fig. 1. Combinations of scenarios are possible but not shown.

We start by examining the workload which we aim to run. In our case, this is conjugate gradient (CG) found in HPCG or variants of similar computations on sparse matrices, wherein the matrices and vectors are normally composed of floating-

point numbers². We model the type of computation performed by looking at memory access patterns as well as the retired instructions. Since sparse matrix algorithms using the CRS representation first reference the row array and then use that data for indexing into the value and columns array, we examine the effects of SDC on this access pattern as illustrated in Fig. 4. If the rows array is corrupted (as depicted in Fig. 4a), then the number of seen elements per row changes, resulting in a change in the position of a value in the matrix. Note that some types of errors in this array are easy to determine, as this array must be sorted in non-decreasing order (by definition). An error in the columns array affects the result of the vector product operation, as it changes which multiplicand is used for a particular member of the vector (Fig. 4b). Lastly, errors in the values array change the result of an operation (Fig. 4c).

When looking at the effects of these errors in the microarchitecture, we see that errors in the rows array induce changes in the access patterns of the values and columns arrays. Moreover, the loop behavior changes as the strides, which are used to determine row information, have changed. Memory access patterns change as a result of the corruption of the rows and columns vectors. Corruption of the rows array implies that column and value information for a particular element is used in the wrong iteration of the algorithm. Corruption of the columns array implies that the wrong vector element is selected when computing the vector product for an element. Therefore, corruption of these arrays has an indirect effect on the overall result of the computation. Lastly, corruption of the values vector has a more direct effect on the correctness of the result of the computation.

A key observation is that counters relevant to memory accesses, branch behavior, and floating point events yield information about the computation graph and can indicate deviation from expected behavior. These counters are given emphasis when considering the underlying microarchitectural events being observed in relation to the high-level code execution. The events which are tracked are dependent on the microarchitecture and can be obtained through analysis of the platform and the PMU subsystem. At a high level, loops are utilized in order to traverse through matrix rows and perform computations. From a microarchitectural perspective, this has a direct effect on the behavior of the branch predictor and branch target buffers in the CPU. Changes in memory access patterns affect cache behavior within the CPU cores, as matrix and/or vector elements will be accessed in different order. Finally, corrupted matrix entries have a direct result on the instructions retired by the floating point units of the CPU, inducing different microarchitectural events, which are then captured by the appropriate event counters. We will discuss what specific hardware counters contribute the most to the model decision boundary when detecting SDC in Section IV-B1.

²Precision of the floating-point numbers or usage of fixed point numbers do not have a direct effect on the methodology. As will be explained later, PMU events are chosen with respect to the workload. Without loss of generality, for the remainder of this section and paper, we use floating-point numbers as the contents of the sparse matrix.

B. Towards a SDC Detection Framework

Interfacing with the PMU can be done through two main mechanisms: using the kernel's `perf` subsystem, or by directly writing to model-specific registers. We limit the runtime overhead of our approach by using the latter. We discuss the advantages and disadvantages of these mechanisms as follows.

Using the `perf` subsystem has the advantage of abstracting away the management of userspace and kernel interactions in order to obtain hardware counters. Moreover, the subsystem provides managed scheduling schemes whenever the number of available counters is less than the number of events to be monitored. However, this comes at the expense of a higher runtime overhead due to an increased number of context switches.

Alternatively, some architectures, such as AMD64, provide access to the PMU hardware directly from a user process without kernel intervention, eliminating the penalty of a context switch. User processes are responsible for scheduling PMU events to account for the limited number of counters. Direct userspace access also presents an issue in systems that make use of preemptive multitasking. As the operating system preempts user processes by interrupting execution at a given quantum, it becomes difficult to decide when to detach the counters from the monitored events. This may result in collecting events from unrelated processes, which creates a noisy dataset for classification. However, userspace access provides the benefit of being able to fine-tune which events are monitored for different portions of the workload. For example, when portions of the workload are loop-dominated the counters can be tied to events relevant to loop behavior.

PMU measurements collected throughout program execution are used as input to a pre-trained model, which can detect whether or not the data in a workload has been corrupted. The model is trained using PMU measurements collected from a diverse set of workloads. Some of these workloads have been synthetically corrupted using the methodology in [34]. Various levels of error injection are utilized to build the training dataset. Corrupted matrices and non-corrupted matrices are used as inputs to the HPCG benchmark. PMU measurements are averaged over multiple runs using the same inputs to account for scheduling variance. We then utilize the trained classifier as part of our framework. To allow for checkpoint computations in our framework, the classifier can be employed at any time to detect corruption at any stage of the computation. If the classifier detects corruption, the computation can then be corrected using e.g. a checkpoint-restart mechanism. Note that error correction is beyond the scope of this work.

C. PMU Justification

To validate our methodology, we utilize a synthetic benchmark of artificially corrupted matrices where we track error propagation on the matrix as iterations on the computations are performed. For this, we run a small experiment where we create sparse matrices and inject errors on them. We then dynamically track the propagation of errors on these matrices

as the computations take place. Fig. 5 showcases an example of 400×400 matrices with the same sparsity coefficient of 0.9975 and a single error injected. Matrices are shown by the random seed used to generate them³. We iterate the SpMV multiplication algorithm on these matrices while collecting feedback from the PMU subsystem.

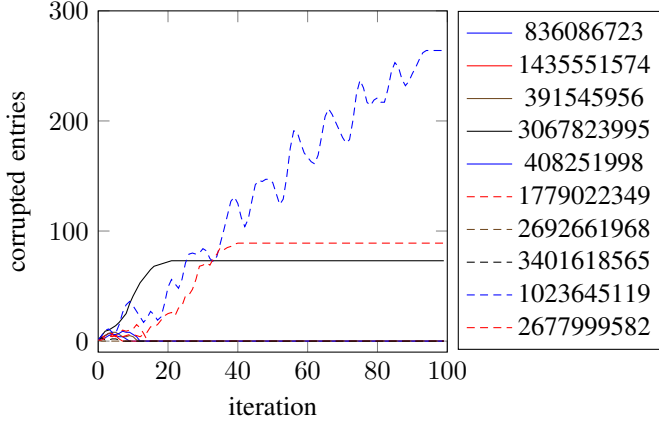


Fig. 5: Error propagation on sparse matrices. Initial position of the error affects the overall propagation of the error.

From this experiment, we conclude two main points. First, the location of the error injected with respect to the non-zero values in the matrix has a direct effect on the propagation of the error in the computation. In some cases, the error disappears, while in other cases the number of erroneous values remains bounded, while in others the number of erroneous values increases with every iteration. Second, that our initial analysis of which performance monitoring counters are affected by error propagation is correct. There is a direct correlation between the value of the counters and the error being propagated.

IV. EXPERIMENTAL EVALUATION

A. Experimental Setup

1) *Hardware Configuration:* We conduct our experiments using CloudLab [18], which provides bare-metal access and control over a substantial set of computing and networking resources. We provision a server with a custom image that closely resembles a typical HPC environment, with two Intel Xeon Silver 4114 10-core CPUs clocked at 2.20 GHz and 192 GiB of ECC DDR4-2666 RAM running Ubuntu 22.04.5 LTS with kernel version 5.15.0-122-generic. Note that ECC protections available in DRAM are insufficient for detecting data corruption caused by a malfunctioning CPU, as ECC assumes that the data is correct during parity calculation and can only detect corruption in either the data or parity afterward. Corruptions that occur before parity is computed will go undetected, since parity is calculated using corrupted data [16], [52].

³Our code for generation, error injection, and SpMV is available at <https://github.com/swson/ADSP>.

2) *Benchmarks:* We evaluate our methodology using the SpMV kernel that utilizes sparse matrices in CSR (Compressed Sparse Row) format and the High Performance Conjugate Gradient (HPCG) benchmark [17], a well-studied benchmark suite used to test the memory subsystem of an HPC system. HPCG generates a 3-dimensional grid of size $n_x \times n_y \times n_z$, where each grid point $p = (x, y, z)$ depends on its own value along with that of 26 surrounding points (27-point stencil). HPCG is designed to be I/O bound, as problem sizes are too large to fit in the processor cache and must be fetched from the main memory. This makes the benchmark a good candidate for error injection. The basic operations executed within HPCG include sparse matrix-vector multiplication (SpMV), vector updates, global dot products, local symmetric Gauss-Seidel smoother, sparse triangular solve, and multigrid (MG) preconditioned conjugate gradient (CG) algorithms. We primarily evaluate the main CG kernel in HPCG, which iteratively executes SpMV, WAXPY ($r = b - Ax$), and dot-product, where A , b , x denote the known system matrix (sparse), the known right-hand side vector, and the initial guess, respectively. To see the variation of PMCs on real data, we added HPCG experiments with sparse matrices from the SuiteSparse Matrix Collection instead of synthetic matrices, as shown in Table I.

3) *Error Injection:* To investigate the effect of silent data corruption, we modify HPCG to support runtime error injection by introducing random noise into the nonzero entries of the input matrix. We employ a single-bit flip, as its occurrence can be more frequent. However, it also introduces significantly more chaotic noise due to the nature of IEEE 754 representation. As we discussed before, such high error rates can be easily detected by users or through simple checks. Instead, we focus on the error models based on our field data presented in Section III-A. Algorithm 1 shows the process of error injection prior to an HPCG run. The symmetric, positive definite matrix A has 27 non-zero entries per row for the interior and 8 – 18 non-zeros for boundary equations. We define the injection rate r to be the fraction of nonzeros injected with errors, and the error rate σ to be the standard deviation of the Gaussian sampler. We experiment with values within the range $r \in [0.01, 0.1]$, $\sigma \in [0.1, 1]$ to generate a training dataset of perturbed matrices.

While the evaluated error range, which can be as little as 10% of deviation (in terms of σ) on 1% of data elements (in terms of r) in the evaluated sparse matrices, may not precisely reflect the observed SDC patterns, errors beyond this range are unlikely to manifest silently. The lowest corruption rate (1%) in our evaluation is even lower than the experimental evaluation used in AID [15], which is as low as 2%, although AID evaluates a different set of matrix benchmarks. More significant errors due to bit flips, particularly in the exponent field, have been previously evaluated and are likely to be captured by the numerical solver as such a corruption makes too large a deviation. Errors less than the evaluated range can be regarded as simply noise and are ultimately masked by the nature of iterative algorithms.

TABLE I: Summary of the matrix datasets used for HPCG experiments. The real-world matrices are selected from the SuiteSparse Matrix Collection, while the HPCG synthetic matrices are generated using a structured 27-point stencil pattern.

Matrix	Description	Total Nodes ($n_x \times n_y \times n_z$)	Rows	Columns	Sparsity	Type
494_bus.mtx	Electric power system, power network problem	-	494	494	0.993	Real-world
662_bus.mtx	Electric power system, power network problem	-	662	662	0.994	Real-world
can_62.mtx	Mechanical structures problems in aircraft design	-	62	62	0.943	Real-world
bcspr03.mtx	Power grid benchmark, power networks pattern	-	118	118	0.965	Real-world
HPCG (grid-4)	Structured 3D grid	$4 \times 4 \times 4$	64	64	0.994	Synthetic
HPCG (grid-8)	Structured 3D grid	$8 \times 8 \times 8$	512	512	0.994	Synthetic
HPCG (grid-16)	Structured 3D grid	$16 \times 16 \times 16$	4,096	4,096	0.994	Synthetic

Algorithm 1 Modified HPCG Algorithm with error injection.

Input: n $\triangleright$ Number of runs
Input: σ, r $\triangleright$ Error parameters
Require: $A \in \mathbb{R}^{n \times m}$
Require: $b, x, y \in \mathbb{R}^n$
 $A \leftarrow \text{NewMatrix}(b, x, y)$
 $A \leftarrow \text{Error}(A, \sigma, r)$ $\triangleright$ Inject error in the matrix
while $n \neq 0$ **do**
 $x \leftarrow 0$ $\triangleright$ Start x at all zeros
 $A \leftarrow \text{CG}(A, b, x)$ $\triangleright$ Conjugate Gradient
 $n \leftarrow n - 1$
end while

4) *Classification Algorithms*: We train the following classifiers to distinguish corrupted and non-corrupted runs from system-wide performance counters. Since we try to classify the collected datasets into two groups: normal (non-corrupted) and abnormal (corrupted), there is a potential issue with imbalanced data, which refers to a situation where the number of samples in the minority class (non-corrupted runs) is much smaller than in the majority class (corrupted runs). This can lead to issues, such as misclassification of minority class instances and over-fitting to the majority class, which may result in poor predictive performance on unseen data. In the generated dataset, the dataset is skewed towards the class of corrupted matrix runs. The standard way of addressing this issue is to apply random oversampling [25] to the training dataset. This technique increases the size of the training set as random sampling involves duplicating a random subset of the minority class (runs with no corrupted data) until it matches the size of the majority class. However, randomly oversampling datasets with a significant class imbalance may lead to duplicating the minor class, which leads to overfitting and does not generalize well to new data. In our experiments, we run the benchmark without corruption ($\sigma = 0, r = 0$) multiple times so that HPCG generates a new problem for every run. The result is a balanced dataset with no loss of generalization.

a) *Logistic Regression (LR)*: Logistic regression is a binary classification algorithm that models the probability of the majority class using the logistic function $\sigma(t) = 1/(1+e^{-t})$, which is parameterized by the linear combination $t = \beta_0 + \beta_1 x$. We estimate the parameters β using maximum likelihood estimation (MLE).

b) *K-Nearest Neighbor (k-NN)*: k-NN assigns an instance to the class that is most common among the k samples nearest to that instance in the feature space. We choose $k = 4$ as this experimentally strikes a good balance between overfitting (too small k) and underfitting (too large k).

c) *Decision Tree (DT)*: The decision tree is a nonparametric supervised learning method that uses a hierarchical structure to make decisions based on input features. The tree is constructed by recursively partitioning the dataset into subsets, reducing the Gini impurity $G = 1 - \sum_i^k p_i$ in each subset, where p_i represents the probability of samples belonging to class i at a given node.

d) *Gradient Boosting (GB)*: Gradient Boosting is an ensemble method that combines the results of multiple decision trees. This method builds trees sequentially, with each tree correcting the errors of the previous one.

e) *Random Forest (RF)*: Random forest is another ensemble method that combines the results of multiple decision trees to reach a single result. Each tree is built independently in parallel using a random subset of the training data. Predictions are made by taking the class selected by the majority of individual trees.

f) *Multi-Layer Perception (MLP)*: A multilayer perceptron (MLP) is a supervised learning algorithm that learns the relationship between an input dataset $\mathbf{X} = (x_1, x_2, \dots, x_m)$ and a target y . MLP includes one or more nonlinear layers, called hidden layers, between the input and output layers (in this case, two hidden layers are used). Through repeated training, the MLP adjusts its weights to fit complex patterns. We set the maximum number of training iterations (max_iter) to 1,000.

g) *Restricted Boltzmann Machine (RBM)*: The Restricted Boltzmann Machine is an *unsupervised* learning algorithm that captures patterns and structures in input data by extracting features. This method transforms the original input space into a set of latent features that emphasize important patterns, after which Logistic Regression (LR) is applied to perform the final classification.

B. Evaluation Results

We evaluate our experiment by comparing the performance of each classifier using k -fold cross-validation. The training data is split into k folds. Each model is trained using $k - 1$ of the folds as training data. The resulting model is validated on the remaining part of the data. To assess classifier performance under practical data conditions and to mitigate potential

Dataset	Classifier	Accuracy	Precision	Recall	F_1 Score
494_bus	LR	0.816	0.874	0.754	0.801
	DT	0.926	0.944	0.910	0.924
	GB	0.928	0.953	0.906	0.925
	RF	0.741	0.761	0.710	0.720
	k-NN	0.820	0.891	0.738	0.791
	MLP	0.843	0.874	0.814	0.837
	RBM+LR	0.597	0.631	0.630	0.567
662_bus	LR	0.614	0.613	0.612	0.605
	DT	0.886	0.886	0.890	0.886
	GB	0.906	0.924	0.890	0.904
	RF	0.729	0.753	0.684	0.706
	k-NN	0.761	0.808	0.688	0.726
	MLP	0.683	0.688	0.662	0.665
	RBM+LR	0.609	0.630	0.592	0.560
bcspwr03	LR	0.629	0.641	0.610	0.620
	DT	0.923	0.962	0.884	0.913
	GB	0.939	0.974	0.904	0.930
	RF	0.946	0.976	0.916	0.938
	k-NN	0.722	0.735	0.698	0.707
	MLP	0.659	0.654	0.724	0.672
	RBM+LR	0.624	0.625	0.626	0.602
can_62	LR	0.593	0.629	0.532	0.563
	DT	0.894	0.929	0.856	0.884
	GB	0.920	0.954	0.886	0.913
	RF	0.920	0.962	0.878	0.909
	k-NN	0.689	0.693	0.694	0.688
	MLP	0.639	0.683	0.584	0.610
	RBM+LR	0.605	0.667	0.454	0.528
grid-4	LR	0.892	0.967	0.817	0.881
	DT	0.970	0.997	0.943	0.968
	GB	0.967	0.997	0.937	0.964
	RF	0.886	0.909	0.857	0.879
	k-NN	0.864	0.927	0.793	0.846
	MLP	0.935	0.983	0.887	0.930
	RBM+LR	0.600	0.612	0.680	0.608
grid-8	LR	0.871	0.907	0.836	0.862
	DT	0.971	0.980	0.964	0.971
	GB	0.962	0.979	0.946	0.960
	RF	0.872	0.919	0.784	0.831
	k-NN	0.882	0.964	0.794	0.855
	MLP	0.941	0.986	0.894	0.936
	RBM+LR	0.705	0.682	0.792	0.721
grid-16	LR	0.895	0.925	0.824	0.867
	DT	0.947	0.934	0.922	0.927
	GB	0.953	0.948	0.918	0.932
	RF	0.882	0.913	0.796	0.841
	k-NN	0.901	0.944	0.814	0.866
	MLP	0.929	0.944	0.874	0.906
	RBM+LR	0.666	0.618	0.700	0.647

TABLE II: Classification performance metrics across different datasets and classifiers. RBM is used for feature extraction, followed by classification with LR.

overfitting to a single benchmark, we performed experiments using both synthetic matrices generated by HPCG and real-world sparse matrices such as `494_bus.mtx`. Furthermore, to ensure sufficient variability and robustness in the measurements, hardware performance counters were collected 100 times under error-free conditions for each dataset. Table II shows the score of each classifier across various key metrics.

1) *Model Performance*: The third column in Table II show the average detection accuracy and F_1 score for each trained

	SW Change	Avg. Recall	% Overhead
AID [15]	FTI library	0.84	6.3
MACORD [48]	state variables	0.95	5
NN-SDC [51]	none	0.90	195.1
This work (with DT)	none	0.91	2

TABLE III: Performance of our approach compared to AID, MACORD, and NN-SDC. Average recall is aggregated across all evaluated applications, as each evaluated approach compares different sets of applications. Overhead is defined as the percentage increase in runtime with the detector enabled.

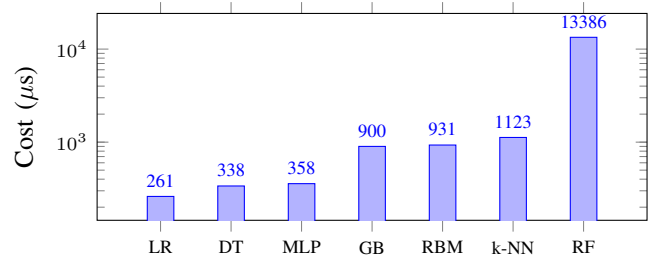


Fig. 6: Average inference overhead, sorted in ascending order, for each classifier across 10^6 full HPCG runs. Each classifier has a negligible runtime (less than 1 s) compared to the runtime of the actual workload.

classifier. Decision Trees outperform the remaining methods, with similar accuracy as the more expensive ensemble methods (Gradient Boosting and Random Forest). The performance of logistic regression (low detection accuracy) suggests that a nonlinear relationship exists between the event counters and the log-likelihood of silent data corruption. This observation also reaffirms the findings of a previous study [23], where a weaker correlation between SDC and performance counters in GPUs is presented. The performance of k-NN, which is based on a distance metric, suggests a lack of spatiotemporal correlation in silent data corruptions that manifest in input data. This k-NN performance potentially indicates why prior approaches that rely on spatial data features produce lower average recall scores compared to our models. Multi-Layer Perception is a neural architecture well-suited for capturing nonlinear relationships, but it demonstrated relatively low detection accuracy in the real-world datasets (i.e., 662_bus, bcspwr03, and can_62) in Table II. This may be attributed to the model architecture (such as the number of hidden layers or the activation functions) not being fully optimized with the characteristics of the detection task. The unsupervised method, RBM, combined with logistic regression (the lowest detection accuracy) may have failed to extract meaningful latent features from the data effectively.

We also show the average precision, recall, and F_1 scores of each tested classifier. Our classifiers display very high precision, showing that PMU counters can yield extremely high-quality data with a low false positive rate when detecting SDC in sparse matrix operations. The high recall of our classifiers further indicates increased detection sensitivity,

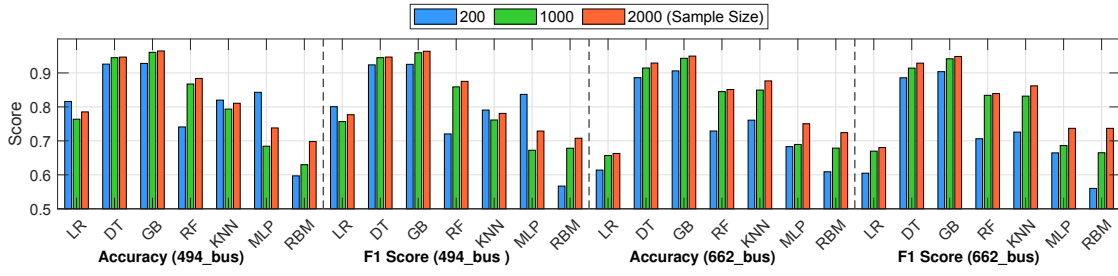


Fig. 7: Classifier performance comparison with varying sample sizes on 494_bus and 662_bus datasets.

where small changes in counter values are enough to detect anomalous CPU behavior, resulting in the minimization of false negatives. Lastly, the presented F_1 scores indicate a good balance between precision and recall. This combination of low false positives and high detection sensitivity suggests a well-rounded model that can effectively detect SDC.

We compare our results to other state of the art application-level approaches in Table III. Application level approaches have the advantage of being readily deployable in existing infrastructure without any changes to the underlying hardware and minor to medium changes to the software stack. AID [15] requires the workload to be modified to make calls to the FTI library to track data. MACORD [48] requires users to manually instrument the workload to indicate which variables should be tracked and protected. NN-SDC [51] is closest to our approach in terms of feature collection, as no modification to the workload is required to extract blocks of processed data. However, it exhibits lower average recall and much higher performance overhead. Our methodology shows improved detection sensitivity compared to the state of the art and requires no modification or instrumentation to the underlying software. Compared to other approaches, collecting PMU data during execution is the least invasive, making our method the most practical for deployment.

We next discuss which specific performance counters contribute the most to the model's decision boundary when detecting SDC, to see whether they clearly capture the behavior of memory access patterns and FP operations. Our experiments utilize the Intel Skylake microarchitecture, which provides eight programmable counters (with hyperthreading disabled) and three fixed counters: `INSTR_RETIRED_ANY`, `CPU_CLK_UNHALTED.THREAD`, and `CPU_CLK_UNHALTED.REF_TSC`. These measure retired instructions, clock cycles at the current frequency, and clock cycles of the reference clock while in the unhalting state, respectively. `IDQ.ALL_DSB_CYCLES_4_UOPS` counts the number of cycles during which the Decode Stream Buffer (DSB) delivers four uops. Variance in this counter is due to the DSB alignment of 32. The `ICACHE_16B.1FDATA_STALL` counter tracks cycles in which a code line fetch is stalled due to an L1 instruction cache miss. The `FP_ARITH_INST_RETIRED.SCALAR_SINGLE` counter tracks the number of floating-point arithmetic instructions retired by the processor's scalar (non-vector) execution

units. This counter is directly affected by the floating-point calculations used in sparse matrix operations. The behaviour of the counters directly align with the preliminary results presented in Section III-A.

2) *Overhead*: The runtime cost of our methodology is a combination of the cost of collecting event counters and the cost of inference. We limit the former by avoiding the Linux `perf` subsystem and collecting raw performance counters from hardware registers directly. Writing to PMU registers (`wrmsr`) requires a context switch to the kernel. Reading hardware counters, however, can be done from a user process with the `rdpmc` instruction. The overhead of collecting event counters accounts for $< 2\%$ of overall execution time. The bulk of our overhead comes from the cost of running the collected counters through a pre-trained model. Inference cost can be controlled by configuring the classifier used for detection. Fig. 6 shows the inference overhead of each classifier across 10^6 independent runs as measured by CPU time. We also record the training (fit) time for each classifier across different datasets. The lowest is approximately 0.02 to 0.03 seconds (LR and DT), while the highest is around 2 seconds (MLP). The third column in Table III shows the runtime overhead of our approach compared to AID, MACORD, and NN-SDC. Our approach achieves lower runtime overhead ($< 2\%$) compared to MACORD (5%), AID (6.3%), and NN-SDC (195.1%). The main reason for this improvement is the use of lightweight performance counters, which are set up once during initialization and do not subsequently interrupt the control flow of the benchmark. Our proposed approach also does not incur high memory overhead as in AID [15], which incurs as high as 15% with optimization, since our approach does not rely on inspecting individual data for learning and detecting the occurrence of SDC.

3) *Impact of Dataset Size on Classifier Performance*: Our experimental results indicate that our proposed approach performs well in terms of detection capability, achieving significantly better scores with the regular ML-based classifiers compared to deep-learning-based or unsupervised ones. This is partly due to the lack of training data, a common bottleneck for supervised learning models. While obtaining unlabeled samples is time-consuming and may hinder the deployment of the proposed approach as an online method, it serves to illustrate how ML model performance can be affected by data.

To investigate how classifier performance varies with the

size of the real-world datasets, we expanded the number of samples from the original 200 (100 normal and 100 abnormal) to 1,000 and 2,000. They show improved performance in terms of accuracy and F-1 score, indicating that a large dataset contributes to better generalization. However, the increase in dataset size also led to a corresponding rise in computational cost. Specifically, both the fit time (training time) and score time (inference time) increased, with GB showing a particularly sharp rise in fit time. This is mainly because GB has a repetitive structure and involves more complex computations than simpler models. As shown in Fig. 7, unlike the other datasets, the 494_bus dataset shows that both LR and MLP achieved higher performance with 200 samples than with larger datasets (i.e., 1,000 and 2,000). This result may stem from the fact that LR, as a simple linear model, performs poorly when the dataset includes more complex and diverse patterns on large datasets that cannot be captured linearly. MLP, with its large number of parameters and nonlinear layers, is prone to overfitting on small datasets.

C. Discussion on Implications of the Evaluation

Variations in performance counter values are observed even when no errors are injected. This is likely due to the sequential structure of computations, where the memory access patterns and cache states of earlier operations can influence subsequent ones, when coupled with general interactions with stochastic events in a computing system, such as interrupts firing in response to network events. Therefore, ensuring independence between operations is necessary. However, fully isolating each computation would incur significant overhead. As a practical compromise, instead of measuring all PMCs, selectively monitoring only meaningful counters can reduce overhead, improving the precision of the results. This approach would also provide a more solid foundation for extending the study toward SDC detection.

The variations in performance counters could also impact the deployment of the proposed approach in practice, as the number of hardware performance counters that can be collected simultaneously is limited, e.g., four to eight in the Intel Skylake microarchitecture used in our evaluated platform. This limited available hardware performance counter remains the same as other CPU models, although each may have different sets of events that users can utilize to measure. Therefore, variations in the most influential performance counters could prevent the proposed approach from being a general mechanism in monitoring and detecting SDCs, as there needs to be a per-data or per-application selection process for utilizing the given fixed number of counters. However, optimizing in this way can be achieved through offline analysis or along with other ML model adjustments, and we will leave these optimizations for future work.

D. Deployment to HPC Environments

Our models can be extended with more datasets and can be seamlessly included in existing HPC runtimes. Hardware PMCs largely go unused in production systems, while mostly

being utilized during development cycles. Our data collection is done through the `perf` subsystem in the Linux kernel. HPC workloads can be instrumented to directly interface with the subsystem, or the `perf` userspace tools can be used instead. For the purposes of this work, we opted for the latter mechanism as it requires no changes to the HPC runtime while showing excellent results. It should be noted that instrumenting the HPC workload to directly interact with the `perf` subsystem can result in more fine-grained data collection, which has the potential of providing even better results.

Instrumentation of an HPC workload can be performed through an instrumenting compiler, such as through the use of custom `#pragma` directives to indicate when data collection should be made, aggregated, and when workloads should be checkpointed. For example, constructs that exhibit looping behavior can be instrumented to collect the behavior of the branch predictor. An instrumented workload can be made to broadcast any data collected from the PMCs to a central node which performs classification to determine the possibility of SDC. Under this scenario, a programmer can add the necessary directives to library and application code as needed, with the instrumenting compiler generating the necessary code to perform the data collection, aggregation, and relay to a classifier node.

V. RELATED WORK

Prior works have studied how SDCs propagate in HPC applications and how to mitigate these issues. For example, Calhoun et al. [7] explored how state alternation due to SDCs traverse at the instruction level using two kernels, including SpMV (similar to ours), and application variable levels using several HPC applications. Their extensive analysis includes detection latency on error propagation and the impact of compiler optimizations and local problem sizes. Di and Cappello [15] introduced a prediction-based detector that adaptively applies the best-fitting model (out of several curve-fitting models) to predict data points and compare them with the observed value. If the difference between the two exceeds a threshold that needs to be provided prior, it will be triggered as an error. Wang et al. [51] investigated errors beyond the impact error bound (i.e., relatively more significant) on iterative applications and proposed a neural-network-based detector that can detect errors propagated through multiple passes. Algorithm-specific detectors also have been proposed [11], [26], [30], [33], which exploit properties of targeted kernels or applications, e.g., fast Fourier transform [33]. While these detectors are effective, they need a significant tailing of applications or high instrumentation or monitoring capability. Our observation can help narrow the scope of vulnerable data or instructions potentially causing immediate or delayed SDCs (or other manifestations leading to ultimate failures), thus significantly mitigating the implementation and instrumentation cost for these approaches.

Given the harmful and dangerous nature of SDCs as they may not be detected by hardware, recent years have witnessed

an increasing adaption of machine learning to predict SDCs in HPC applications [48], [50], [55]. For example, focusing on parallel benchmark programs like LULESH and CoMD, Sirius utilizes neural networks to learn locality patterns and detect SDCs accurately, assuming the existence of spatial and temporal locality within most variables (i.e., data). Yang and Wang [55] used a support vector machine (SVM) classifier to train and detect vulnerable instructions, likely causing SDCs. SVM is a good candidate for capturing spatio-temporal locality in data and is less expensive than a neural network. This approach works well when there is a spatio-temporal correlation in the usage of program variables that might be affected by SDC and errors are injected at the instruction level. However, if the data itself is corrupt, there is no guarantee that errors will be correlated in space or time. Other than these prior works, the most similar to ours is MACORD [48], which compares the approach of similar learning algorithms, however, their focus was primarily on training cost. Nevertheless, these prior techniques presented competitive detection performance using various metrics (e.g., 98% of SDCs with a false positive rate of less than 0.02 in Sirius [50], 89% detection rates with an FPR (false positive rate) of less than 2% in [51], and recall up to 99% with an FPR of 0.1% in [48]) and reasonable overheads associated with their techniques (e.g., about 5% and 1% of computation and memory overheads, respectively, in [48] and up to 29.51x overhead on CPU of the detector with 1.83–2.5 hours of training time in [51]). Traditional column-sum-based ABFT (Algorithm-Based Fault Tolerance) techniques [45], [47] detect errors by leveraging mathematical algorithms such as column checksums and, when necessary, recover results through checksum-based correction. However, this approach suffers from significant computational overhead due to the additional checksum calculations and recovery procedures. Online-ABFT [12] was proposed to overcome the limitations of traditional ABFT. It monitors mathematical properties such as residual reduction and orthogonality to detect errors during computation in real time. As a result, Online-ABFT achieves lower overhead compared to traditional ABFT while enabling effective error detection. However, it still requires extra computation and remains tied to specific algorithms (e.g., CG, SpMV).

VI. CONCLUSION

In this paper, we conducted an extensive investigation of the impact of SDC on sparse matrix computations with comprehensive controlled error injection and measurement. We first characterized the computational aspect of sparse matrix computations, using SpMV in the compressed row storage (CRS) format as an example, and how potential silent data corruption (SDC) could propagate during operation. We then presented our methodology, which collects hardware performance counters and learns the patterns due to SDCs, ultimately capturing latent performance signatures due to the existence of SDCs. Our methodology implemented several optimizations and scheduling to efficiently collect performance counters while maximizing the effectiveness of collected values for

training and inference. Our experimental evaluations using the model built on seven classification algorithms (LR, k-NN, DT, GB, RF, MLP, and RBM) demonstrated our approach's superior performance in all measured metrics of quality and overhead incurred, surpassing prior approaches using machine learning and neural network models. With the model in the decision tree and gradient boosting, our approach can achieve precision, recall, accuracy, and F_1 close to 1 while incurring comparable training overheads and minor inference overheads against the state-of-the-art SDC detection methods.

We aim to expand on this research by: expanding the evaluation of our framework on matrices that are not symmetrical and other broader scientific kernels and applications, varying the error and injection rates beyond the range we evaluated in the paper in order to determine our framework can accurately detect a variety of SDC profiles, introduce other performance monitoring application interfaces instead of Linux perf, particularly PAPI (performance application programming interface) in order to compare hardware counter metrics, and evaluate the efficacy of runtime error correction mechanisms after errors have been detected to determine if these recovery mechanisms have the potential to introduce corruption into the dataset.

ACKNOWLEDGMENT

This material is in part based upon work supported by the National Science Foundation under Grant No. 2312982.

REFERENCES

- [1] H. Alyahya, R. Mehmood, and I. Katib. Parallel sparse matrix vector multiplication on intel mic: Performance analysis. In R. Mehmood, B. Bhaduri, I. Katib, and I. Chlamtac, editors, *Smart Societies, Infrastructure, Technologies and Applications*, pages 306–322, Cham, 2018. Springer International Publishing.
- [2] M. Berry, T. E. Potok, P. Balaprakash, H. Hoffmann, R. Vatsavai, and Prabhat. Machine Learning and Understanding for Intelligent Extreme Scale Scientific Computing and Discovery, 2015. Department of Energy Office of Science.
- [3] B. Bittel, M. Shamsa, B. Inkley, A. Gur, D. Lerner, and M. Adams. Data Center Silent Data Errors: Implications to Artificial Intelligence Workloads & Mitigations. In *2024 IEEE International Reliability Physics Symposium (IRPS)*, pages 1–5, 2024.
- [4] A. Buluç, J. T. Fineman, M. Frigo, J. R. Gilbert, and C. E. Leiserson. Parallel sparse matrix-vector and matrix-transpose-vector multiplication using compressed sparse blocks. In *Proceedings of the twenty-first annual symposium on Parallelism in algorithms and architectures*, pages 233–244, 2009.
- [5] A. Buluç and J. R. Gilbert. Challenges and advances in parallel sparse matrix-matrix multiplication. In *2008 37th International Conference on Parallel Processing*, pages 503–510. IEEE, 2008.
- [6] A. Buluç and J. R. Gilbert. Parallel sparse matrix-matrix multiplication and indexing: Implementation and experiments. *SIAM Journal on Scientific Computing*, 34(4):C170–C191, 2012.
- [7] J. Calhoun, M. Snir, L. N. Olson, and W. D. Gropp. Towards a More Complete Understanding of SDC Propagation. In *Proceedings of the 26th International Symposium on High-Performance Parallel and Distributed Computing*, page 131–142, 2017.
- [8] F. Cappello, G. Al, W. Gropp, S. Kale, B. Kramer, and M. Snir. Toward Exascale Resilience: 2014 Update. *Supercomput. Front. Innov.: Int. J.*, 1(1):5–28, Apr. 2014.
- [9] J. Chen, H. Li, S. Li, X. Liang, P. Wu, D. Tao, K. Ouyang, Y. Liu, K. Zhao, Q. Guan, and Z. Chen. Fault Tolerant One-sided Matrix Decompositions on Heterogeneous Systems with GPUs. In *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 854–865, 2018.

- [10] J. Chen, X. Liang, and Z. Chen. Online Algorithm-Based Fault Tolerance for Cholesky Decomposition on Heterogeneous Systems with GPUs. In *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 993–1002, 2016.
- [11] Z. Chen. Online-ABFT: An Online Algorithm Based Fault Tolerance Scheme for Soft Error Detection in Iterative Methods. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP '13*, page 167–176, 2013.
- [12] Z. Chen. Online-abft: An online algorithm based fault tolerance scheme for soft error detection in iterative methods. *ACM SIGPLAN Notices*, 48(8):167–176, 2013.
- [13] T. A. Davis and Y. Hu. The University of Florida sparse matrix collection. *ACM Trans. Math. Softw.*, 38(1), dec 2011.
- [14] E. Deelman, T. Peterka, I. Altintas, C. D. Carothers, K. K. van Dam, K. Moreland, M. Parashar, L. Ramakrishnan, M. Taufer, and J. Vetter. The Future of Scientific Workflows. *The International Journal of High Performance Computing Applications*, 32(1):159–175, 2018.
- [15] S. Di and F. Cappello. Adaptive Impact-Driven Detection of Silent Data Corruption for HPC Applications. *IEEE Transactions on Parallel and Distributed Systems*, 27(10):2809–2823, 2016.
- [16] H. D. Dixit, S. Pendharkar, M. Beadon, C. Mason, T. Chakravarthy, B. Muthiah, and S. Sankar. Silent data corruptions at scale. *CoRR*, abs/2102.11245, 2021.
- [17] J. Dongarra, P. Luszczek, and M. Heroux. Hpcg technical specification. *Sandia National Laboratories, Sandia Report SAND2013-8752*, 2013.
- [18] D. Duplyakin, R. Ricci, A. Maricq, G. Wong, J. Duerig, E. Eide, L. Stoller, M. Hibler, D. Johnson, K. Webb, A. Akella, K. Wang, G. Ricart, L. Landweber, C. Elliott, M. Zink, E. Cecchet, S. Kar, and P. Mishra. The Design and Operation of CloudLab. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, pages 1–14, July 2019.
- [19] X. Fu, J. Poe, T. Li, and J. Fortes. Characterizing Microarchitecture Soft Error Vulnerability Phase Behavior. In *14th IEEE International Symposium on Modeling, Analysis, and Simulation*, pages 147–155, 2006.
- [20] A. Geist. How to kill a supercomputer: Dirty power, cosmic rays, and bad solder. *IEEE Spectrum*, 2016.
- [21] A. Geist. Supercomputing’s monster in the closet. *IEEE Spectrum*, 53(3):30–35, 2016.
- [22] J. L. Greathouse and M. Daga. Efficient sparse matrix-vector multiplication on GPUs using the CSR storage format. In *SC’14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 769–780. IEEE, 2014.
- [23] S. K. S. Hari, P. Rech, T. Tsai, M. Stephenson, A. Zulfiqar, M. B. Sullivan, P. P. Shirvani, P. Racunas, J. S. Emer, and S. W. Keckler. Estimating Silent Data Corruption Rates Using a Two-Level Model. *CoRR*, abs/2005.01445, 2020.
- [24] P. H. Hochschild, P. Turner, J. C. Mogul, R. Govindaraju, P. Ranganathan, D. E. Culler, and A. Vahdat. Cores that don’t count. In *Proceedings of the Workshop on Hot Topics in Operating Systems, HotOS ’21*, page 9–16, New York, NY, USA, 2021. Association for Computing Machinery.
- [25] T. R. Hoens and N. V. Chawla. *Imbalanced Datasets: From Sampling to Classifiers*, chapter 3, pages 43–59. John Wiley & Sons, Ltd, 2013.
- [26] K.-H. Huang and J. A. Abraham. Algorithm-Based Fault Tolerance for Matrix Operations. *IEEE Transactions on Computers*, C-33(6):518–528, 1984.
- [27] Intel Corporation. <https://perfmon-events.intel.com/>.
- [28] Intel Corporation. *Intel 64 and IA-32 Architectures Performance Monitoring Events*, December 2017.
- [29] E. W. L. Leng, M. Zwolinski, and B. Halak. Hardware performance counters for system reliability monitoring. In *2017 IEEE 2nd International Verification and Security Workshop (IVSW)*, pages 76–81, 2017.
- [30] D. Li, Z. Chen, P. Wu, and J. S. Vetter. Rethinking algorithm-based fault tolerance with a cooperative software-hardware approach. In *SC ’13: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, pages 1–12, 2013.
- [31] S. Li, J. Huang, P. T. P. Tang, D. Khudia, J. Park, H. D. Dixit, and Z. Chen. Efficient Soft-Error Detection for Low-precision Deep Learning Recommendation Models, 2021.
- [32] S. Li, J. Huang, P. T. P. Tang, D. Khudia, J. Park, H. D. Dixit, and Z. Chen. Efficient Soft-Error Detection for Low-precision Deep Learning Recommendation Models. In *2022 IEEE International Conference on Big Data (Big Data)*, pages 1556–1563, 2022.
- [33] X. Liang, J. Chen, D. Tao, S. Li, P. Wu, H. Li, K. Ouyang, Y. Liu, F. Song, and Z. Chen. Correcting Soft Errors Online in Fast Fourier Transform. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC ’17*, New York, NY, USA, 2017. Association for Computing Machinery.
- [34] A. Moon, M. Kim, J. Chen, and S. W. Son. Anomaly Detection in Scientific Datasets using Sparse Representation. In *Proceedings of the First Workshop on AI for Systems, AI4Sys ’23*, page 13–18, 2023.
- [35] National Science Foundation. *Transforming the World Through Science*, 2nd Edition, 2018.
- [36] B. Nie, J. Xue, S. Gupta, C. Engelmann, E. Smirni, and D. Tiwari. Characterizing Temperature, Power, and Soft-Error Behaviors in Data Center Systems: Insights, Challenges, and Opportunities. In *2017 IEEE 25th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, pages 22–31, 2017.
- [37] NVIDIA. *NVIDIA PerfKit User Guide*, June 2008.
- [38] S. Ohshima, T. Katagiri, and M. Matsumoto. Performance optimization of spmv using crs format by considering openmp scheduling on cpus and mic. In *2014 IEEE 8th International Symposium on Embedded Multicore/Manycore SoCs*, pages 253–260. IEEE, 2014.
- [39] S. Pal, J. Beaumont, D.-H. Park, A. Amarnath, S. Feng, C. Chakrabarti, H.-S. Kim, D. Blaauw, T. Mudge, and R. Dreslinski. Outerspace: An outer product based sparse matrix multiplication accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 724–736. IEEE, 2018.
- [40] G. Papadimitriou and D. Gizopoulos. AVGI: Microarchitecture-Driven, Fast and Accurate Vulnerability Assessment. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 935–948, 2023.
- [41] S. Peisert. Security in High-performance Computing Environments. *Commun. ACM*, 60(9):72–80, Aug. 2017.
- [42] S. Peisert, G. Cybenko, S. Jajodia, D. L. Brown, C. L. DeMarco, P. Hovland, S. Leyffer, C. Matarazzo, S. Prowell, B. Tierney, and V. Welch. ASCR Cybersecurity for Scientific Computing Integrity, 2015. Department of Energy Office of Science.
- [43] S. Peisert, T. E. Potok, and T. Jones. ASCR Cybersecurity for Scientific Computing Integrity – Research Pathways and Ideas Workshop, 2015. Department of Energy Office of Science.
- [44] A. Pinar and M. T. Heath. Improving performance of sparse matrix-vector multiplication. In *Proceedings of the 1999 ACM/IEEE conference on Supercomputing*, pages 30–es, 1999.
- [45] A. Schöll, C. Braun, M. A. Kochte, and H.-J. Wunderlich. Efficient algorithm-based fault tolerance for sparse matrix operations. In *2016 46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 251–262. IEEE, 2016.
- [46] R. Shan and J. C. Calhoun. Exploring Data Corruption Inside SZ. In *2022 IEEE International Conference on Big Data (Big Data)*, pages 3172–3178, 2022.
- [47] J. Sloan, R. Kumar, and G. Bronevetsky. An algorithmic approach to error localization and partial recomputation for low-overhead fault tolerance. In *2013 43rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 1–12. IEEE, 2013.
- [48] O. Subasi, S. Di, P. Balaprakash, O. Unsal, J. Labarta, A. Cristal, S. Krishnamoorthy, and F. Cappello. MACORD: Online Adaptive Machine Learning Framework for Silent Error Detection. In *2017 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 717–724, 2017.
- [49] D. Tao, S. L. Song, S. Krishnamoorthy, P. Wu, X. Liang, E. Z. Zhang, D. Kerbyson, and Z. Chen. New-Sum: A Novel Online ABFT Scheme For General Iterative Methods. In *Proceedings of the 25th ACM International Symposium on High-Performance Parallel and Distributed Computing, HPDC ’16*, page 43–55, New York, NY, USA, 2016. Association for Computing Machinery.
- [50] T. E. Thomas, A. J. Bhattad, S. Mitra, and S. Bagchi. Sirius: Neural Network Based Probabilistic Assertions for Detecting Silent Data Corruption in Parallel Programs. In *2016 IEEE 35th Symposium on Reliable Distributed Systems (SRDS)*, pages 41–50, 2016.
- [51] C. Wang, N. Dryden, F. Cappello, and M. Snir. Neural Network Based Silent Error Detector. In *2018 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 168–178, 2018.
- [52] S. Wang, G. Zhang, J. Wei, Y. Wang, J. Wu, and Q. Luo. Understanding Silent Data Corruptions in a Large Production CPU Population. In *Proceedings of the 29th Symposium on Operating Systems Principles*,

- SOSP '23, page 216–230, New York, NY, USA, 2023. Association for Computing Machinery.
- [53] S. Williams, L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel. Optimization of sparse matrix-vector multiplication on emerging multi-core platforms. In *Proceedings of the 2007 ACM/IEEE Conference on Supercomputing*, pages 1–12, 2007.
 - [54] P. Wu, N. DeBardeleben, Q. Guan, S. Blanchard, J. Chen, D. Tao, X. Liang, K. Ouyang, and Z. Chen. Silent Data Corruption Resilient Two-Sided Matrix Factorizations. In *Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP '17*, page 415–427, New York, NY, USA, 2017. Association for Computing Machinery.
 - [55] N. Yang and Y. Wang. Identify Silent Data Corruption Vulnerable Instructions Using SVM. *IEEE Access*, 7:40210–40219, 2019.
 - [56] A. Yasin. A Top-Down method for performance analysis and counters architecture. In *2014 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 35–44, 2014.
 - [57] R. Yuster and U. Zwick. Fast sparse matrix multiplication. *ACM Transactions On Algorithms (TALG)*, 1(1):2–13, 2005.
 - [58] J. Zhang, K. Rangineni, Z. Ghodsi, and S. Garg. ThUnderVolt: Enabling Aggressive Voltage Underscaling and Timing Error Resilience for Energy Efficient Deep Learning Accelerators. In *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*, pages 1–6, 2018.
 - [59] Z. Zhang, H. Wang, S. Han, and W. J. Dally. SpArch: Efficient Architecture for Sparse Matrix Multiplication. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 261–274. IEEE, 2020.
 - [60] L. Zhuo and V. K. Prasanna. Sparse matrix-vector multiplication on FPGAs. In *Proceedings of the 2005 ACM/SIGDA 13th international symposium on Field-programmable gate arrays*, pages 63–74, 2005.